

dles auxiliary variables through syntactic regularization and extracts multiple valid subrules without relying on predefined templates. These results further highlight ANDRE’s robustness, interpretability, and suitability for structured relational reasoning.

G.2 UNSTRUCTURED RULE: ARBITRARY TASK

To demonstrate ANDRE’s internal behavior and learning dynamics, we constructed a controlled synthetic dataset $E \in \mathbb{R}^{100 \times (4+1)}$ consisting of 100 examples. Each row $e \in E$ represents a training sample composed of fuzzy truth values assigned to four potential body predicates $\mathbf{b} = \{b_1, b_2, b_3, b_4\}$, and a corresponding Boolean label $h \in \{0, 1\}$ representing the ground truth. These labels are generated based on a known logical rule composed of two subrules: $h \leftarrow h_1 \vee h_2$, where the subrules are defined as: $h_1 \leftarrow \neg b_2 \wedge b_4$ and $h_2 \leftarrow b_1 \wedge \neg b_3$. The corresponding fuzzy condition for generating Boolean labels is encoded as:

$$h = ((1 - b_2) > 0.5 \wedge b_4 > 0.5) \vee (b_1 > 0.5 \wedge (1 - b_3) > 0.5).$$

The objective for ANDRE is to infer both the correct number of subrules and the structure of each subrule (i.e., which predicates to include, exclude, or negate), using only the probabilistic inputs and Boolean outputs without prior knowledge of the underlying rule structure.

The symbolic subrule matrix $B_{ij}^{\text{andre}} \in \{b_j, \neg b_j, 1\}^{n \times 4}$ is initialized as empty and must be inferred by the model during training, where n is the number of subrules, and $j = 1, \dots, 4$ refers to the candidate predicates.

Training proceeds by feeding probabilistic valuations b into the network, computing the predicted labels $\hat{h}$, and comparing them with the ground-truth labels h . The binary cross-entropy loss $\mathcal{L}_{\text{BCE}}$ and the entropy regularization loss $\mathcal{L}_{\text{E}}$ are used to guide training (see Eq. 10 and Eq. 11). Since this example uses only zero-arity predicates (i.e., propositional inputs), no head or auxiliary variables are present, so all syntactic losses are deactivated in this example. Only semantic losses are active. The total loss is minimized via gradient-based backpropagation.

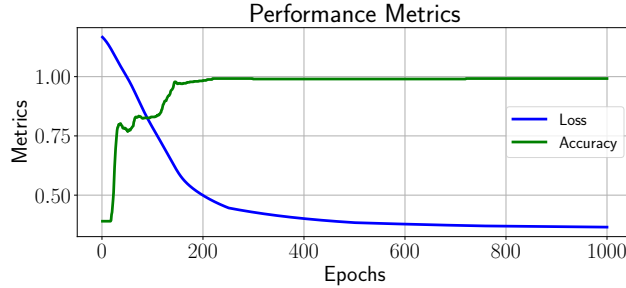


Figure 7: ANDRE performance metrics during training.

Figure 7 illustrates how the loss and accuracy metrics evolve during training. Accordingly, the loss converges to the minimum value during the training, and the training accuracy converged to 1.0, meaning that model could successfully find the global optimum.

Figure 8 depicts the evolution of the softmax-normalized weights across training. For subrule $\hat{h}_1$, the converged subpredicates are: $\{1, \neg b_2, 1, b_4\}$, corresponding to: $\hat{h}_1 \leftarrow \neg b_2 \wedge b_4$. For subrule $\hat{h}_2$, the model converges to: $\{b_1, 1, \neg b_3, 1\}$, which translates to: $\hat{h}_2 \leftarrow b_1 \wedge \neg b_3$. Thus, the final symbolic matrix extracted by ANDRE is:

$$B_{ij}^{\text{andre}} = \begin{bmatrix} 1 & \neg b_2 & 1 & b_4 \\ b_1 & 1 & \neg b_3 & 1 \end{bmatrix}.$$

From this matrix, the overall rule reconstructed by ANDRE is:

$$\hat{h} \leftarrow (\neg b_2 \wedge b_4) \vee (b_1 \wedge \neg b_3),$$

which exactly matches the target rule used to generate the dataset. The total training time was approximately 1.83 seconds, highlighting the efficiency of ANDRE even on small datasets.

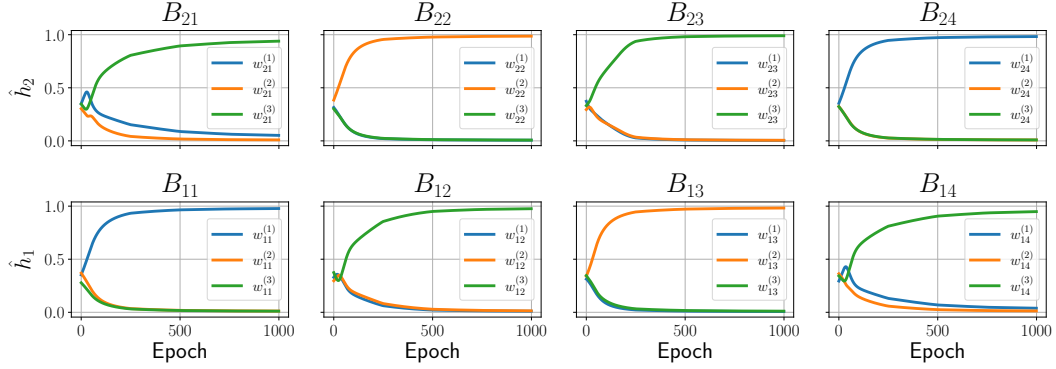


Figure 8: Final subpredicate probabilities for each B_{ij} . Each subplot shows the softmax-normalized probability distribution over three symbolic forms: b_j (blue), $\neg b_j$ (orange), and 1 (green). Sharp convergence indicates high confidence in subpredicate selection.

H ANDRE RESULTS ON CLASSICAL ILP TASKS

This section presents ANDRE’s performance on a set of classical ILP benchmark tasks widely used in the literature. For each task, we provide the formal representation of the data and background structure required for propositionalization, followed by the final symbolic rule extracted by ANDRE after training.

Throughout this section, the following notations are used consistently:

1. X : the complete set of logical variables used in the task.
2. X^h : the subset of variables appearing in the head predicate.
3. $\mathbb{E}$: the domain of possible constant values for the variables in X .
4. $\mathbf{b} = \{b_1, b_2, \dots, b_m\}$: the set of candidate body predicates.
5. h : the symbolic head predicate to be learned.
6. $\mathbb{B}$: the background knowledge containing supporting facts.
7. $\mathcal{P}$: the set of positive examples (grounded instances of the head predicate).
8. $\mathcal{N}$: the set of negative examples.

For each task, we apply a propositionalization procedure to convert the symbolic representations into a trainable dataset. After training ANDRE, we report the discovered symbolic rule, which is expected to generalize over both training and test samples.

1. The Predecessor Task:

Objective: Learn a rule that identifies when one number is the predecessor of another, using background knowledge about numeric succession.

- Variable set: $X = \{X_1, X_2\}$
- Head variable set: $X^h = X$
- Domain of constants: $\mathbb{E} = \{0, 1, \dots, 8\}$
- Body predicates: $\mathbf{b} = \{\text{successor}(X_2, X_1)\}$
- Head predicate: $h = \text{predecessor}(X_1, X_2)$
- Background knowledge:

$$\mathbb{B} = \{\text{successor}(X_i, X_{i+1}) \mid X_i \in \mathbb{E}\} \cup \{\text{zero}(0)\}$$

- Positive examples:

$$\mathcal{P} = \{\text{predecessor}(X_j, X_i) \mid X_j = X_i + 1, X_i \in \mathbb{E}\}$$

- Negative examples:

$$\mathcal{N} = \{\text{predecessor}(X_j, X_i) \mid X_j \neq X_i + 1, X_i \in \mathbb{E}\}$$

After training, the learned rule extracted by ANDRE is:

$$\text{predecessor}(X_1, X_2) :- \text{successor}(X_2, X_1). \quad (19)$$

This result aligns perfectly with the expected inverse of the `successor` relation, demonstrating ANDRE’s ability to infer correct relational structure from structured examples and background knowledge.

2. The Odd Task:

Objective: Learn the logical patterns that define odd numbers using predecessor and parity relationships, and generalize them to unseen numerical values.

- Variable set: $X = \{X_1, X_2, X_3\}$
- Head variable set: $X^h = \{X_3\}$
- Auxiliary variable set: $X^a = X \setminus X^h$
- Domain of constants: $\mathbb{E} = \{0, 1, \dots, 30\}$
- Body predicates:

$$\mathbf{b} = \{\text{zero}(X_1), \text{zero}(X_2), \text{zero}(X_3), \text{successor}(X_1, X_2), \\ \text{successor}(X_2, X_3), \text{successor}(X_1, X_3), \text{odd}(X_1), \text{odd}(X_2)\}$$

- Head predicate: $\mathbf{h} = \text{odd}(X_3)$
- Background knowledge:

$$\mathbb{B} = \{\text{successor}(X_i, X_i + 1) \mid X_i \in \mathbb{E}\} \cup \{\text{zero}(0)\}$$

- Positive examples:

$$\mathcal{P} = \{\text{odd}(X_i) \mid X_i \bmod 2 = 1, X_i \in \mathbb{E} \cup \{31\}\}$$

- Negative examples:

$$\mathcal{N} = \{\text{odd}(X_i) \mid X_i \bmod 2 = 0, X_i \in \mathbb{E}\}$$

After applying the propositionalization method, the training dataset E is constructed and ANDRE is trained. On unseen test data, the extracted subrules are:

1. $\text{odd}(X_3) :- \text{zero}(X_2), \text{successor}(X_2, X_3).$
2. $\text{odd}(X_3) :- \text{successor}(X_2, X_3), \neg \text{odd}(X_2).$
3. $\text{odd}(X_3) :- \text{successor}(X_2, X_3), \text{successor}(X_1, X_2), \text{odd}(X_1).$

3. The Even Task:

Objective: Discover the rule structure that governs even numbers, exploiting arithmetic successor relationships and parity predicates.

- Variable set: $X = \{X_1, X_2, X_3\}$
- Head variable set: $X^h = \{X_3\}$
- Auxiliary variable set: $X^a = X \setminus X^h$
- Domain of constants: $\mathbb{E} = \{0, 1, \dots, 30\}$
- Body predicates:

$$\mathbf{b} = \{\text{zero}(X_1), \text{zero}(X_2), \text{zero}(X_3), \text{successor}(X_1, X_2), \\ \text{successor}(X_2, X_3), \text{successor}(X_1, X_3), \text{even}(X_1), \text{even}(X_2)\}$$